

Advanced Hooks

Learning Objectives

Today we will learn:

- useRef
- useMemo
- useCallback
- Component Re-rendering
- Performance Optimization
- Parent and Child Performance

Easy Understanding

useRef ↓ Store a value without re-rendering

useMemo ↓ Remember a calculated value

useCallback ↓ Remember a function

❏ **Important:** These hooks are mainly used for **optimization and special use cases**. Do not use them everywhere.

useRef, What is it?

useRef()

useRef is used to store a value that persists between renders without causing a re-render.

Syntax

```
const valueRef = useRef();
```

Access the value using:

```
valueRef.current
```

Most Common Uses

- Access DOM elements
- Focus an input
- Store timer ID
- Store previous value
- Store mutable values

useState vs useRef

useState

Value changes ↓ Re-render

useRef

Value changes ↓ No re-render

useRef, Complete Example

Focus Input

```
import { useRef } from "react";
function App() {
  const inputRef = useRef();
  const focusInput = () => {
    inputRef.current.focus();
  };
  return (
    <>
      Focus
    </>
  );
}
export default App;
```

Step by Step

01

Create Ref

```
const inputRef = useRef();
```

02

Attach Ref

03

Access Element

```
inputRef.current
```

04

Call DOM Method

```
inputRef.current.focus();
```

📌 **Real World:** Login forms, search boxes, modals, video controls and timers.

useMemo, What is it?

useMemo()

useMemo remembers the result of a calculation.

It is useful when a calculation is expensive and should not run unnecessarily.

Syntax

```
const result = useMemo(() => {  
  return calculation();  
}, [dependencies]);
```

Easy Way to Remember

useMemo ↓ Memoize a VALUE

Example

```
const result = useMemo(() => {  
  return products.filter(product =>  
    product.name.includes(search)  
  );  
}, [search]);
```

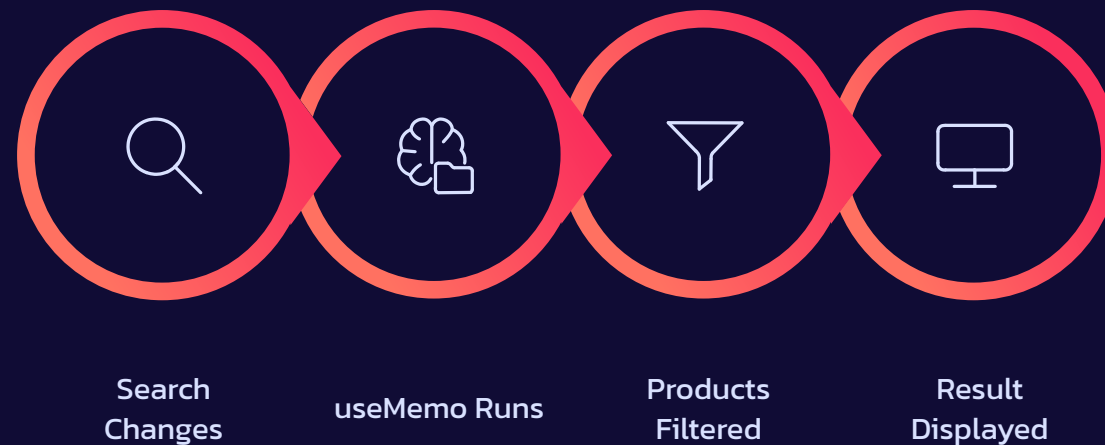
The filtering runs again when search changes.

useMemo, Search Example

Product Search Optimization

```
import { useMemo, useState } from "react";
function App() {
  const [search, setSearch] = useState("");
  const products = [
    "Laptop", "Mobile", "Mouse", "Keyboard"
  ];
  const result = useMemo(() => {
    return products.filter(item =>
      item.toLowerCase()
        .includes(search.toLowerCase())
    );
  }, [search]);
  return (
    <>
      <input
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        placeholder="Search"
      />
      {result.map(item => (
        <div>{item}</div>
      ))}
    </>
  );
}
export default App;
```

Flow



useCallback, What is it?

useCallback()

useCallback remembers a function between renders.

Syntax

```
const handleClick = useCallback(() => {  
  // logic  
}, []);
```

Easy Way to Remember

useMemo ↓ Remember a VALUE

useCallback ↓ Remember a FUNCTION

Why is this useful?

When a function is passed from a parent to a child, a new function can be created whenever the parent renders.

useCallback can keep the same function reference.

useCallback, Parent and Child

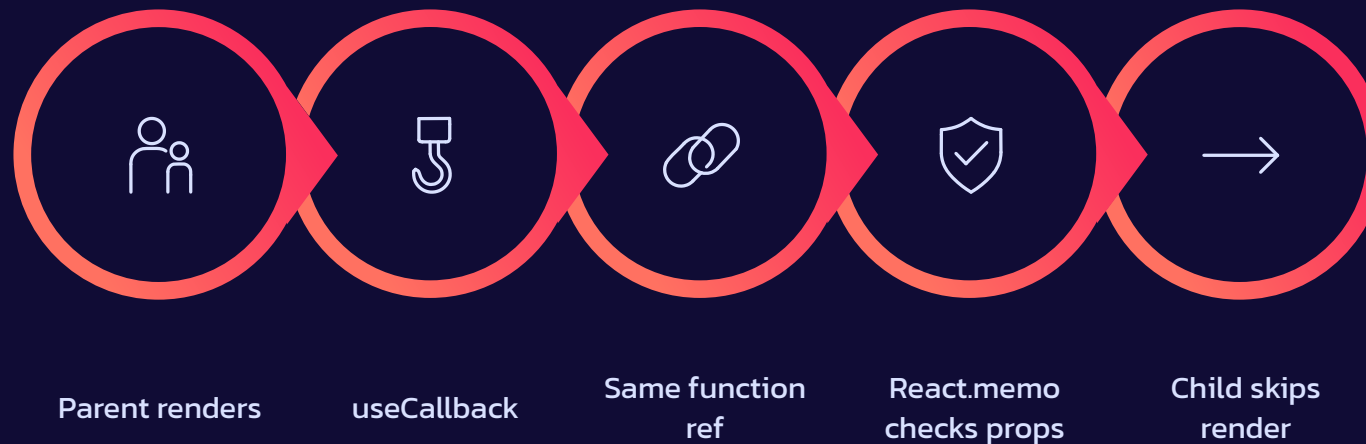
```
import { memo, useCallback, useState } from "react";
const Child = memo(({ onClick }) => {
  console.log("Child Rendered");
  return (
    <button onClick={onClick}>Add Product</button>
  );
});
function App() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => {
    console.log("Product Added");
  }, []);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Count: {count}
      </button>
      <Child onClick={handleClick} />
    </>
  );
}
export default App;
```

Important Concepts

useCallback()
Keeps the function reference stable.

memo()
Allows the child to skip unnecessary rendering when its props have not changed.

Flow



useRef vs useMemo vs useCallback

Hook	Purpose	Stores
useRef	Persistent value or DOM access	Mutable value
useMemo	Avoid expensive calculations	Calculated value
useCallback	Keep function reference stable	Function

Easy Memory Trick

useRef

"What should I remember without rendering?"

useMemo

"What calculation should I remember?"

useCallback

"What function should I remember?"

Real World Examples

- useRef → Input focus → Timer
- useMemo → Product filtering → Expensive calculation
- useCallback → Parent → Child functions → Performance optimization

Mini Task

Build 3 Small Features

1

Stopwatch

Use: `useState` `useRef`

Features:

- Start
- Stop
- Reset
- Display seconds

2

Search Optimization

Use: `useState` `useMemo`

Features:

- Product search
- Filter products
- Display results

3

Parent Child Performance Demo

Use: `React.memo` `useCallback`

Features:

- Parent counter
- Child button
- Console render message
- Compare behavior before and after optimization

Student Experiment

First remove `useCallback`.

Then add it again.

Observe:

```
console.log("Child Rendered");
```

This helps students understand the actual performance difference.

Homework, Optimized Product Search

Build an Optimized Product Search

Requirements

Create a product search application using:

- useState
- useMemo
- useCallback
- React.memo

Features

- Search products
- Filter products
- Display Bootstrap Product Cards
- Add Product button
- Product count
- Optimize ProductCard rendering

Product Card

Display:

- Product Image
- Product Name
- Price
- Category
- Add to Cart button

Expected Flow

Search ↓ useState ↓ useMemo ↓ Filtered Products ↓ ProductCard ↓ useCallback ↓ Add Product

Submission Checklist

1 ✓ useRef	2 ✓ useMemo	3 ✓ useCallback
4 ✓ React.memo	5 ✓ useState	6 ✓ filter()
7 ✓ Reusable Components		

Final Takeaway

→ useRef → DOM and persistent values

→ useMemo → Cached calculated values

→ useCallback → Cached functions

→ React.memo → Avoid unnecessary child renders

📌 **Goal:** Students should understand not only how these hooks work, but also **why and when they should be used** in real React applications.